

Red Local para el desarrollo de programas WEBXR usando el casco MetaQuest 3

Versión definitiva

Requerimiento ir al directorio en donde se encontrará FASTAPI e instalar:

```
pip install cryptography uvicorn fastapi
```

Configuración del desarrollo raíz

Nombre	Fecha de modificación	Tipo	Tamaño
__pycache__	15/08/2026 01:00 p. m.	Carpeta de archivos	
Equipo01	15/08/2026 12:14 p. m.	Carpeta de archivos	
Equipo02	15/08/2026 12:14 p. m.	Carpeta de archivos	
Equipo03	18/09/2026 02:26 p. m.	Carpeta de archivos	
Equipo04	18/09/2026 02:26 p. m.	Carpeta de archivos	
Instructor	18/09/2026 02:26 p. m.	Carpeta de archivos	
dns_local	26/09/2026 04:03 p. m.	Certificado de seg...	2 KB
dns_local	26/09/2026 04:03 p. m.	Presentación de A...	2 KB
index	18/09/2026 02:45 p. m.	Chrome HTML Do...	4 KB
Iniciar_MetaQuest	26/09/2026 10:30 a. m.	Archivo por lotes ...	1 KB
main.py	26/09/2026 04:07 p. m.	Python.File	6 KB

main.py

```
import datetime
import os
import socket
import ipaddress
import uvicorn
from fastapi import FastAPI
from fastapi.staticfiles import StaticFiles
from fastapi.middleware.cors import CORSMiddleware
from cryptography import x509
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import rsa
from cryptography.x509.oid import NameOID

#
=====

# CONFIGURACION DEL ENTORNO LOCAL PARA META QUEST
#
=====

# Direccion IP fija asignada en el DNS de tu router TP-Link
IP_ROUTER = "192.168.0.100"
PUERTO_SERVICIO = 8443

def generar_certificado_dns_fijo():
    """Genera certificados SSL locales con soporte SAN correcto para Chromium."""
    cert_file = "dns_local.crt"
    key_file = "dns_local.key"
```

```
if not os.path.exists(cert_file) or not os.path.exists(key_file):
```

```
    print(f"[INFO] Generando certificado SSL robusto para la IP: {IP_ROUTER}...")
```

```
# 1. Generar la llave privada RSA
```

```
private_key = rsa.generate_private_key(
```

```
    public_exponent=65537,
```

```
    key_size=2048
```

```
)
```

```
# 2. Configurar los detalles del emisor (Subject/Issuer)
```

```
subject = issuer = x509.Name([
```

```
    x509.NameAttribute(NameOID.COUNTRY_NAME, "MX"),
```

```
    x509.NameAttribute(NameOID.STATE_OR_PROVINCE_NAME, "Qro"),
```

```
    x509.NameAttribute(NameOID.ORGANIZATION_NAME, "Universidad"),
```

```
    x509.NameAttribute(NameOID.COMMON_NAME, IP_ROUTER),
```

```
])
```

```
# 3. PASO CRITICO: Configurar extensiones SAN (Subject Alternative Name)
```

```
# Separacion estricta exigida por Chromium: Dominios en DNSName e IPs en IPAddress
```

```
alt_names = [
```

```
    x509.DNSName("localhost"),
```

```
    x509.DNSName("127.0.0.1"),
```

```
    x509.IPAddress(ipaddress.IPv4Address(IP_ROUTER)) # Correccion de tipado estricto  
para Chromium
```

```
]
```

```
# 4. Construir el certificado valido por 1 aoo
```

```
cert = (
```

```
    x509.CertificateBuilder()
```

```
        .subject_name(subject)
```

```

.issuer_name(issuer)
.public_key(private_key.public_key())
.serial_number(x509.random_serial_number())
.not_valid_before(datetime.datetime.now(datetime.timezone.utc))
.not_valid_after(
    datetime.datetime.now(datetime.timezone.utc) + datetime.timedelta(days=365)
)
.add_extension(
    x509.SubjectAlternativeName(alt_names), critical=False
)
.add_extension(
    x509.ExtendedKeyUsage([x509.oid.ExtendedKeyUsageOID.SERVER_AUTH]),
    critical=False,
)
.sign(private_key, hashes.SHA256())
)

```

5. Guardar la llave privada en el disco duro (.key)

with open(key_file, "wb") as f:

```

f.write(
    private_key.private_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PrivateFormat.TraditionalOpenSSL,
        encryption_algorithm=serialization.NoEncryption(),
    )
)

```

6. Guardar el certificado en el disco duro (.crt)

with open(cert_file, "wb") as f:

```

f.write(cert.public_bytes(serialization.Encoding.PEM))

```

```

    print("[OK] Certificados SSL creados con exito.")

    return cert_file, key_file

#
=====

# CONFIGURACION DE FASTAPI (IDENTICA A TU VERSION DE CELULARES)
#
=====

app = FastAPI()

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Monta la raiz de la carpeta actual de manera identica a la solucion http de los telefonos
app.mount("/", StaticFiles(directory=".", html=True), name="static")

if __name__ == "__main__":
    # Obtener las rutas de los certificados seguros
    cert_path, key_path = generar_certificado_dns_fijo()

    # Validar la existencia fisica de las llaves antes de arrancar el servidor
    if os.path.exists(cert_path) and os.path.exists(key_path):
        print(f"\n[SERVIDOR] Iniciando entorno seguro para Meta Quest en:
        https://{IP_ROUTER}:{PUERTO_SERVICIO}")

```

```
print("[INSTRUCCION] En el casco abre Meta Quest Browser e ingresa exactamente la URL de arriba.\n")
```

```
# Arrancamos Uvicorn forzando HTTPS nativo a traves de la red local inalambrica
uvicorn.run(
    "main:app",
    host="0.0.0.0",
    port=PUERTO_SERVICIO,
    ssl_keyfile=key_path,
    ssl_certfile=cert_path,
    reload=False # Desactivado para evitar bucles infinitos en dispositivos de almacenamiento
    externos (USB)
)
else:
    print("[ERROR] Archivos de certificados SSL invalidos o no encontrados en la raiz.")
```

Iniciación del sistema

Paso 1: Encender la infraestructura inalámbrica local

1. Enciende tu router TP-Link Archer GE230 (no necesita estar conectado al cable de la universidad, solo a la corriente).
2. Conecta tu laptop Alienware m16 R1 a la red Wi-Fi de ese router. Asegúrate en la configuración de Windows de que tu laptop tenga asignada la IP fija `192.168.0.100`.
3. Ve a tu carpeta `ClaseWebXRMeta` en la unidad Kingston y dale doble clic al archivo `Iniciar_MetaQuest.bat`.
 - *Qué debes ver:* Se abrirá la terminal en verde. Si es la primera vez, verás el mensaje de que se crearon los archivos `dns_local.crt` y `dns_local.key`. Luego, verás que FastAPI y Uvicorn quedan parpadeando, listos y esperando conexiones en el puerto 8443.

Paso 2: Conectar el casco Meta Quest 3

1. Colócate el casco y entra a la configuración de red. Conéctalo al Wi-Fi de tu router TP-Link.
2. Abre la aplicación Meta Quest Browser (el navegador nativo del visor).
3. En la barra de direcciones escribe exactamente: `https://192.168.0.100:8443`

Paso 3: Forzar la excepción de seguridad (Paso único para el Instructor)

1. Al cargar la página por primera vez, el casco te mostrará una pantalla de advertencia gris o roja ("*La conexión no es privada*" o "*Certificado no válido*").
2. Con tu control, haz clic en el botón "Configuración avanzada" (o "Detalles") en la parte inferior.
3. Selecciona "Acceder a 192.168.0.100 (sitio no seguro)".
4. ¡Listo! Verás el menú "QueretaFLARE". Entra a **Equipo01/proy01/**, dale clic al botón "START AR" en la parte inferior y la nave y el leñador aparecerán en tu propia habitación con una nitidez espectacular y sin dientes de sierra.